MODULE 08 — FINAL

Real-World React TS Patterns

The patterns that separate a React JS developer from a React TypeScript engineer. API layers, typed forms, generic components, compound components, and a complete project architecture.

~65 min read | All modules required | Production ready

1. Building a Typed API Layer

The most impactful thing you can do in a React TS project is build a strongly-typed API layer. Every component that fetches data benefits when the types flow from the server response all the way to the JSX:

api/types.ts

```ts
// api/types.ts — all your API types in one place
export interface User    { id: number; name: string; email: string; role:
'admin' | 'user'; }
export interface Post    { id: number; title: string; body: string;
authorId: number; }
export interface ApiError { message: string; code: number; }

export type ApiResult<T> =
  | { ok: true;  data: T }
  | { ok: false; error: ApiError };
```

api/client.ts and api/users.ts

```ts
// api/client.ts — generic typed fetch wrapper
import { ApiResult, ApiError } from './types';

const BASE_URL = 'https://api.example.com';

async function apiRequest<T>(
  path:    string,
  options?: RequestInit
```

```
): Promise<ApiResult<T>> {
  try {
    const res  = await fetch(`${BASE_URL}${path}`, {
      headers: { 'Content-Type': 'application/json' },
      ...options,
    });
    if (!res.ok) {
      const error: ApiError = await res.json();
      return { ok: false, error };
    }
    const data: T = await res.json();
    return { ok: true, data };
  } catch (e) {
    return { ok: false, error: { message: String(e), code: 0 } };
  }
}

// api/users.ts — typed API functions, one per resource
import type { User } from './types';

export const usersApi = {
  getAll:  ()                       => apiRequest<User[]>('/users'),
  getById: (id: number)             => apiRequest<User>(`/users/${id}`),
  create:  (data: Omit<User, 'id'>) => apiRequest<User>('/users',
{ method: 'POST', body: JSON.stringify(data) }),
  update:  (id: number, data: Partial<User>) => apiRequest<User>(`/users/$
{id}`, { method: 'PATCH', body: JSON.stringify(data) }),
  delete:  (id: number)             => apiRequest<void>(`/users/${id}`,
{ method: 'DELETE' }),
};

// Usage in a component
const result = await usersApi.getById(1);
if (result.ok) {
  console.log(result.data.name);  // result.data is User — fully typed
} else {
  console.error(result.error.message);  // error path is typed too
}
```

2. Typed Forms — react-hook-form + Zod

The industry standard for typed forms in React TS is react-hook-form with Zod for schema validation. Zod infers TypeScript types from your validation schema — one source of truth for both validation rules and types:

Terminal — install

```
npm install react-hook-form zod @hookform/resolvers
```

react-hook-form + Zod

```
import { useForm }         from 'react-hook-form';
import { zodResolver }     from '@hookform/resolvers/zod';
import { z }              from 'zod';

// 1. Define schema with Zod — this IS your validation + your type
const loginSchema = z.object({
  email:    z.string().email('Invalid email'),
  password: z.string().min(8, 'Min 8 characters'),
  remember: z.boolean().optional(),
});

// 2. Infer the TypeScript type from the schema — no duplication
type LoginFormData = z.infer<typeof loginSchema>;
// { email: string; password: string; remember?: boolean }

function LoginForm() {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<LoginFormData>({ resolver: zodResolver(loginSchema) });

  // onSubmit receives fully-typed, validated data
  const onSubmit = async (data: LoginFormData): Promise<void> => {
    // data.email is a valid email string — Zod guarantees it
    await usersApi.login(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('email')} type='email' />
      {errors.email && <span>{errors.email.message}</span>}
      <input {...register('password')} type='password' />
      {errors.password && <span>{errors.password.message}</span>}
      <button disabled={isSubmitting}>Login</button>
    </form>
  );
}
```

> **Why Zod?**
>
> Zod validates at runtime (catching bad API responses, user inputs) AND generates TypeScript types at compile time. One schema does both jobs. This is the single-source-of-truth principle applied to validation — change the schema and both your runtime checks and compile-time types update automatically.

3. Generic Components — Reusable and Type-Safe

Generic components are one of the most powerful React TS patterns. A generic List component that works for any data type is far more useful than one that only works for User[]:

Generic List<T> component

```
// A generic List component — works for any array of objects with an id
interface ListProps<T extends { id: number | string }> {
  items:      T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor?: (item: T) => string;
  emptyMessage?: string;
}

function List<T extends { id: number | string }>({
  items, renderItem, emptyMessage = 'No items', keyExtractor
}: ListProps<T>) {
  if (items.length === 0) return <p>{emptyMessage}</p>;
  return (
    <ul>
      {items.map(item => (
        <li key={keyExtractor ? keyExtractor(item) : String(item.id)}>
          {renderItem(item)}
        </li>
      ))}
    </ul>
  );
}

// Usage — T is inferred from items
interface User    { id: number; name: string; email: string; }
interface Product { id: number; name: string; price: number; }

<List
  items={users}                              // T = User
  renderItem={(user) => <span>{user.name} — {user.email}</span>}
  // user is typed as User — autocomplete works!
/>
```

```
<List
  items={products}                          // T = Product
  renderItem={(p) => <span>{p.name}: ${p.price}</span>}
  emptyMessage='No products available'
/>
```

4. Compound Components Pattern

Compound components share implicit state through Context. Think of how HTML <select> and <option> work together. In React TS, you type the shared context and each sub-component:

Compound Tabs component

```
// A typed Tabs compound component

interface TabsContextType {
  activeTab: string;
  setActiveTab: (tab: string) => void;
}

const TabsContext = createContext<TabsContextType | null>(null);
const useTabs = () => {
  const ctx = useContext(TabsContext);
  if (!ctx) throw new Error('Must be inside <Tabs>');
  return ctx;
};

// Root component
function Tabs({ children, defaultTab }: { children: React.ReactNode;
defaultTab: string }) {
  const [activeTab, setActiveTab] = useState(defaultTab);
  return <TabsContext.Provider value={{ activeTab, setActiveTab }}
>{children}</TabsContext.Provider>;
}

// Sub-components — each typed independently
function Tab({ id, label }: { id: string; label: string }) {
  const { activeTab, setActiveTab } = useTabs();
  return <button onClick={() => setActiveTab(id)} aria-selected={activeTab
=== id}>{label}</button>;
}

function TabPanel({ id, children }: { id: string; children:
```

```
React.ReactNode }) {
  const { activeTab } = useTabs();
  return activeTab === id ? <div>{children}</div> : null;
}

// Attach sub-components as static properties — clean API
Tabs.Tab      = Tab;
Tabs.TabPanel = TabPanel;

// Usage — self-contained, fully typed, no prop drilling
<Tabs defaultTab='profile'>
  <Tabs.Tab id='profile' label='Profile' />
  <Tabs.Tab id='settings' label='Settings' />
  <Tabs.TabPanel id='profile'>  <ProfileContent />  </Tabs.TabPanel>
  <Tabs.TabPanel id='settings'> <SettingsContent /> </Tabs.TabPanel>
</Tabs>
```

5. Polymorphic Components — The 'as' Prop

A polymorphic component renders as different HTML elements based on an 'as' prop. This is how design system components work — a Button that can render as an <a> tag or a <button>:

Polymorphic Text component

```
import { ComponentPropsWithoutRef, ElementType, ReactNode } from 'react';

// The magic type — combines element-specific props with your own
type PolymorphicProps<T extends ElementType> = {
  as?:      T;         // which element/component to render as
  children: ReactNode;
} & ComponentPropsWithoutRef<T>;  // include that element's native props

function Text<T extends ElementType = 'span'>({
  as,
  children,
  ...rest
}: PolymorphicProps<T>) {
  const Component = as ?? 'span';
  return <Component {...rest}>{children}</Component>;
}

// Renders as <span> — TS allows span props
<Text>Hello</Text>
```

```tsx
// Renders as <h1> — TS allows h1 props
<Text as='h1' id='page-title'>Page Title</Text>

// Renders as <a> — TS allows anchor props including href
<Text as='a' href='https://example.com'>Click me</Text>

// Renders as <a> but missing href — TS doesn't error (href is optional)
// But passing wrong prop IS an error:
<Text as='span' href='...' />  // Error: href not on span
```

6. Recommended Project Structure

A well-organized React TS project makes types discoverable and maintainable. Here's the structure used in production codebases:

Project structure

```
src/
  api/
    client.ts          # generic apiRequest<T> wrapper
    types.ts           # all API interfaces (User, Product, etc)
    users.ts           # usersApi object with typed methods
    products.ts        # productsApi ...
  components/
    ui/                # generic reusable: Button, Input, Card
      Button.tsx
      Input.tsx
    features/          # feature-specific: UserCard, ProductList
      users/
        UserCard.tsx
        UserList.tsx
  hooks/               # custom hooks: useFetch, useDebounce
    useFetch.ts
    useLocalStorage.ts
    useDebounce.ts
  store/               # context + reducer stores
    auth.tsx            # AuthProvider, useAuth
    cart.tsx            # CartProvider, useCart
  types/               # shared type aliases
    index.ts           # re-exports all shared types
  pages/              # route-level components
    HomePage.tsx
    ProfilePage.tsx
  App.tsx
```

```
  main.tsx
tsconfig.json
vite.config.ts
```

7. Essential tsconfig.json Settings

The TypeScript compiler is configured through tsconfig.json. These are the settings that matter most for a React project:

tsconfig.json — recommended settings

```
{
  "compilerOptions": {
    "target": "ES2020",          // JS version to compile to
    "lib": ["ES2020", "DOM"],   // available APIs (DOM = browser)
    "jsx": "react-jsx",           // React 17+ JSX transform
    "module": "ESNext",
    "moduleResolution": "bundler",

    // Strictness — enable ALL of these in new projects
    "strict": true,               // enables all strict checks
    // strict: true enables these (no need to list them separately):
    // strictNullChecks    — null/undefined must be handled explicitly
    // noImplicitAny       — can't use untyped variables
    // strictFunctionTypes — function param types must match exactly

    // Extra strictness — highly recommended
    "noUnusedLocals": true,        // error on unused variables
    "noUnusedParameters": true,    // error on unused function params
    "noFallthroughCasesInSwitch": true, // switch must handle all cases
    "exactOptionalPropertyTypes": true,  // optional props can't be set to
undefined

    // Paths alias — import from '@/components' instead of '../../'
    "baseUrl": ".",
    "paths": { "@/*": ["./src/*"] }
  }
}
```

8. Common React TS Gotchas & How to Fix Them

These are the errors every developer hits when moving from React JS to React TS:

Gotcha 1 — 'Object is possibly null'

Null safety

```
// Problem: TS doesn't know if user is null
const [user, setUser] = useState<User | null>(null);
return <h1>{user.name}</h1>;  // Error: user is possibly null

// Fix 1: Optional chaining
return <h1>{user?.name}</h1>;

// Fix 2: Conditional render (preferred)
if (!user) return <p>Loading...</p>;
return <h1>{user.name}</h1>;  // TS knows user is User here
```

Gotcha 2 — 'Type string | undefined is not assignable to string'

Optional prop safety

```
// Problem: optional prop used where required
interface Props { name?: string; }  // optional
function greet({ name }: Props) {
  return name.toUpperCase();  // Error: name might be undefined
}

// Fix: provide a fallback
return (name ?? 'Guest').toUpperCase();
// or:
return (name || 'Guest').toUpperCase();
```

Gotcha 3 — 'Type never' in exhaustive switches

Exhaustive switch

```
// Problem: forgot a case in switch — TS is right to complain!
type Status = 'loading' | 'success' | 'error';
function render(s: Status) {
  switch (s) {
    case 'loading': return <Spinner />;
    case 'success': return <Data />;
    // forgot 'error'! TS: function lacks return type
  }
```

```
}

// Fix: handle all cases
case 'error': return <ErrorMessage />;
```

Gotcha 4 — Event handler type mismatch

Event handler wrapping

```
// Problem: passing raw function where event handler expected
function saveUser(user: User) { ... }
<button onClick={saveUser}>Save</button>  // Error: User vs MouseEvent

// Fix: wrap in arrow function
<button onClick={() => saveUser(currentUser)}>Save</button>
```

You've Completed the Full Course

You now have the complete React TypeScript skill set. Here's everything you've mastered across all 8 modules:

- Module 1 — The TypeScript type system: primitives, unions, type aliases
- Module 2 — Typing functions, objects, interfaces, extends
- Module 3 — Generics: write once, use for any type
- Module 4 — Enums, narrowing, type guards, discriminated unions
- Module 5 — Utility types: Partial, Omit, Pick, ReturnType and more
- Module 6 — React fundamentals: props, events, useState, useRef
- Module 7 — Hooks and Context: useReducer, useContext, custom hooks
- Module 8 — Production patterns: API layer, forms, generic components

C O U R S E C O M P L E T E

React TypeScript Engineer

You went from zero TypeScript experience to production-grade React TS patterns across 8 modules. Your JS expertise is now supercharged with the type safety, autocomplete, and refactoring confidence that TypeScript provides.

What to do next:

-> Pick an existing React JS project and migrate it to TypeScript file by file
-> Start a new project with Vite react-ts template from day one
-> Add react-hook-form + Zod to a form-heavy project
-> Explore TanStack Query (React Query) — it is built for TypeScript